

Année Universitaire 2025 – 2026
Unité de Recherche : IPGP
Équipe : Dynamique des fluides géologiques
Tuteur de stage : F. Métivier
Enseignant référent : I. Martinez



Développement et optimisation d'un système de mesure automatisé en Python

VELASCO Mathis

Université Paris Cité, IPGP, Janson de Sailly

Du 26/05/2026 au 06/07/2026

Résumé

Ce rapport présente la conception et l'optimisation du programme d'un appareil mesurant une température à l'aide d'une sonde thermique PT100 et d'une interface *MAX31865* d'Adafruit ainsi qu'un potentiel d'oxydo-réduction (ORP) en utilisant une sonde ORP. Il présente d'abord quelques généralités sur le potentiel d'oxydo-réduction, puis comment fonctionne le système Arduino du boîtier de l'appareil ainsi que la calibration à un et deux étalons qui peut être appliquée à la sonde ORP. Ensuite, les tests de reproductibilité menés sur sept sondes ont montré une excellente cohérence inter-sonde dans les solutions tampons tandis que la forte influence des solutions étalons sur la reproductibilité des résultats dans l'eau déminéralisée a souligné l'importance d'une calibration adaptée à la plage de potentiel du milieu étudié. Le programme Python **SAFE-M-ORP** réalisé est disponible publiquement sur <https://github.com/SAFE-M/SAFE-M-ORP-.git>.

Mots-Clés : Sonde ORP, calibration, Python, Arduino, SAFE-M

Sommaire

Résumé	1
1 Introduction	2
2 La sonde ORP, qu'est-ce que c'est ?	2
2.1 Principe d'Oxydo-réduction	2
2.2 Utilisation des sondes ORP	3
3 Influence des composants	3
4 Montage de l'appareil	4
4.1 Le boîtier	4
4.2 Arduino	4
4.3 Connexion boîtier / Ordinateur	5
5 Calibration	6
5.1 L'équation d'oxydo-réduction :	6
5.2 Calibration d'usine et à 1 étalon	6
5.3 Calibration à deux étalons	6
6 Les mesures	8
6.1 Acquisition des Mesures	8
6.2 Tests des sondes ORP	8
6.2.1 Méthode d'analyse	8
6.2.2 Discussion des résultats	8
6.2.3 Reproductibilité dans l'eau déminéralisée	9
7 Conclusion	10
8 Remerciements	10
A Annexes	12
A.1 Organisation du code	12
A.2 Utilisation de l'IA	13
A.3 Quelques exemples de fonction du projet SAFE-M-ORP	14

1 Introduction

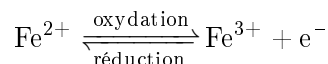
Ce stage s'est déroulé à l'Institut de Physique du Globe de Paris (IPGP) dans le service de dynamique des fluides géologiques. Il a consisté à développer et automatiser numériquement un appareil de mesure d'une température et de potentiel d'oxydo-réduction. Cet appareil est le prolongement d'une Unité d'Enseignement (UE) nommée *Mesures automatisées en physique et chimie de l'environnement* de notre sixième semestre de la licence de Sciences de la Terre. Cette UE, dirigée par M. Métivier, a permis la première conception d'un boîtier de mesure et d'un code permettant de lire les mesures d'un capteur de température et du potentiel d'oxydo-réduction des électrodes de nos sondes. L'objectif de ce stage était donc d'optimiser et de restructurer le code Python existant, d'améliorer l'interface utilisateur pour une meilleure clarté et compréhension ainsi que l'export et l'organisation des données mesurées.

Les enjeux de ce stage ont été premièrement de réaliser un code clair et compréhensible, pour que les boîtiers soient donnés à des enseignants malgaches dans le cadre du projet SAFE-M (*Soutenir l'Apprentissage et les Formations sur l'Eau à Madagascar*). Ensuite des questions sur les sondes ORP se sont posées : dans quel milieu pouvons-nous les utiliser, les mesures sont-elles précises et reproductibles ? La découverte et l'utilisation des outils Git, Github et Python utilisés ne sont pas détaillées ici mais le projet est déposé publiquement sur : <https://github.com/SAFE-M/SAFE-M-ORP-.git>

2 La sonde ORP, qu'est-ce que c'est ?

2.1 Principe d'Oxydo-réduction

L'ORP décrit la tendance d'une espèce chimique à gagner des électrons (réduction) ou à en perdre (oxydation). **Exemple :**



On considère une demi-pile par la demi-équation :

$$\alpha_1 \text{Ox} + \alpha_2 \text{A} + n_{e-} = \alpha_3 \text{Red} + \alpha_4 \text{B} \quad (1)$$

où α_i est le coefficient stœchiométrique ; A et B sont les auxiliaires permettant d'équilibrer l'équation ; Ox est l'oxydant présent en solution ; Red est le réducteur présent en solution.

Avec chaque couple d'oxydant-réducteur qui suit, à l'équilibre électrochimique (courant électrique nul, $i=0$), la relation de Nernst :

$$E = E_{\text{Ox/Red}}^\circ + \frac{RT}{nF} \log \left(\frac{a_{\text{Ox}}^{\alpha_1} \cdot a_{\text{A}}^{\alpha_2}}{a_{\text{Red}}^{\alpha_3} \cdot a_{\text{B}}^{\alpha_4}} \right) \quad (2)$$

où $8,31 \text{ J K}^{-1} \text{ mol}^{-1}$ est la constante des gaz parfaits ; T la température ; n le nombre d'électrons échangés ; $F = N_a \cdot e = 96485 \text{ C mol}^{-1}$ la constante de Faraday, avec N_a le nombre d'Avogadro et e la charge élémentaire ; $a_{\alpha_i}^i$ est l'activité des espèces mises en jeu ; enfin $E_{\text{Ox/Red}}^\circ$ est le potentiel standard du couple d'oxydo-réduction étudié à $\text{pH} = 0$.

Cette relation permet de mettre en évidence que si le potentiel augmente, c'est l'oxydant qui prédomine, et inversement. Quand le potentiel augmente, le pouvoir oxydant augmente, et inversement. On pourra donc caractériser des milieux plutôt oxydants ou plutôt réducteurs. La mesure de l'ORP est réalisée grâce à une cellule électrochimique constituée d'une électrode de mesure et d'une électrode de référence.

2.2 Utilisation des sondes ORP

Ces sondes ont une application importante dans le traitement des eaux usées. En effet, l'ORP d'une eau usée est lié à l'ensemble des espèces oxydantes (oxygène dissous, nitrates) et réductrices (ammonium, matière organique) qui la compose. Des études [5] ont montré une corrélation positive entre l'ORP, la présence de nitrate et l'oxygène dissous, et négative avec l'azote ammoniacal et le pH, faisant de l'ORP un indicateur indirect de la qualité de l'eau.

Elles peuvent également servir en hydrogéochemie pour caractériser les zones de transitions entre les eaux souterraines et les sédiments [6]. En effet, à l'aide de carotte sédimentaire l'ORP permet d'établir un profil vertical montrant la séparation des milieux par la présence de minéraux ferreux.

Les sondes réalisées lors de ce stage seront utilisées lors de travaux pratiques pour l'étude de la qualité de l'eau (la potabilité et pollution).

3 Influence des composants

Expérimentalement, on ne peut mesurer qu'une différence de potentiel, par convention il a été convenu de prendre comme référence que le potentiel de l'électrode standard à hydrogène (ESH) est nul quelle que soit la température. Ainsi nous pouvons calculer le potentiel à l'aide d'une cellule électrochimique en faisant circuler du courant à travers deux électrodes : l'une plongée dans sa solution dont le potentiel est connu (électrode de référence) et l'autre dans la solution désirée (électrode de mesure).

Avec l'aide d'un potentiomètre, on peut ainsi lire une valeur de potentiel.

Nos sondes ORP sont constituées d'une électrode de mesure en platine et/ou or et d'une électrode de référence Ag/AgCl, le système Arduino (voir 4.2) jouant le rôle de potentiomètre.

Le document de B.Striggow ([3]) nous donne un exemple reprenant un montage (1b) très similaire à nos sondes pour mieux comprendre l'utilisation de ce référentiel. En effet, le potentiel de l'électrode Ag/AgCl référencé par rapport à l'ESH renvoie 204 mV¹ (1a) tandis que le potentiel d'une ESH dans l'échantillon renvoie 544 mV et l'électrode Ag/AgCl renvoie 340 mV ($340 + 204 = 544\text{mV}$). Donc l'utilisation et le choix de l'électrode de référence va induire un potentiel mesuré différent :

- 340 mV pour Ag/AgCl
- 544 mV pour ESH

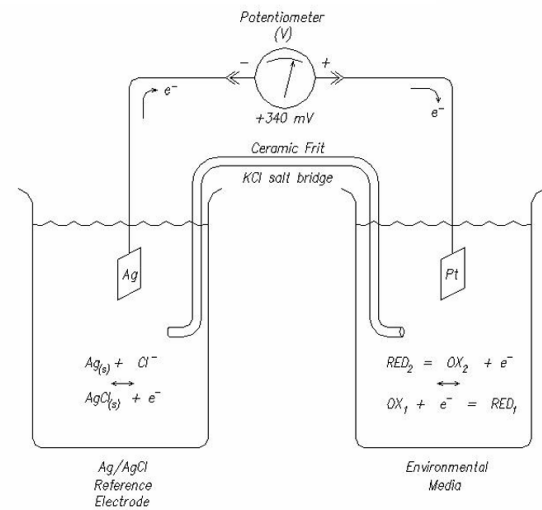
Ainsi, le résultat d'une mesure de potentiel n'a de sens que si l'on connaît la référence utilisée. En pratique, nous ramenons souvent, et par la suite, les potentiels par rapport à ESH noté E_h .

1. L'auteur ne précise pas avec quelle molarité il a utilisé son électrode mais on suppose qu'il a pris Sat/4M à 20°C car il note plutôt qu'elle est souvent utilisée.

Half-cell Potential of Ag/AgCl reference electrode
derived from USGS NFM, Table 6.5.2 (9/2005)

T(°C)	Molarity of KCl filling solution			
	3M	3.3M*	3.5M	Sat/4M
10	220	217	215	214
15	216	214	212	209
20	213	210	208	204
25	209	207	205	199
30	205	203	201	194
35	202	199	197	189
40	198	195	193	184

(a)



(b)

FIGURE 1 – Tableau de référence des variations de potentiel d'une électrode Ag/AgCl en fonction de sa molarité (mol.L^{-1}) et de la température ($^{\circ}\text{C}$). (b) Schéma d'une cellule électrochimique d'une électrode Ag/AgCl fonctionnant avec une électrode de mesure en platine et un pont salin en céramique. ([3], voir p.9, 2.2, Figure 3 et Table 1).

4 Montage de l'appareil

4.1 Le boîtier

Les composants de l'appareil :

- sintron box : coffret plastique 100 x 68 x 50 mm
- carte Arduino Uno R3
- sonde de température PT100
- interface MAX31865 d'Adafruit
- sonde ORP
- joint silicone
- fil de câblage
- boulon avec sa couronne
- amplificateur de la sonde

Ce boîtier est présenté dans la figure 2b.

4.2 Arduino

La figure 2a est une représentation schématique du système Arduino installé sur les appareils. La carte Arduino est un circuit électrique connecté à un ordinateur par câble USB. La figure 2a présente deux rangées de connecteurs composées de :

- 6 entrées analogiques qui permettent de recevoir des signaux analogiques afin de mesurer des tensions.
- 12 entrées ou sorties digitales
- 2 ground terre
- 2 connecteurs de Puissance (deux sorties : 3V3, 5V)
- Une alimentation (VIN)

Les connecteurs digitaux vont permettre à la carte de convertir ces tensions mesurées en un signal numérique encodé par un nombre de 10 bits qui renvoie des entiers entre 0 et 1023 traitables par l'ordinateur.

Avec Arduino, nous avons téléversé un programme codé en langage C permettant de mesurer la température et la tension de la solution dans laquelle les capteurs sont plongés. On a utilisé les fonctions :

```
1 void setup() {
2 //Fonction mesurant la tension
3   Serial.begin(115200);
4 //Fonction mesurant la température grâce à l'interface MAX31865 d'Adafruit.
5   thermo.begin(MAX31865_2WIRE);
6 }
```

Après avoir connecté l'Arduino à l'ordinateur (voir 4.3), cette tension est lue sur Python avec la bibliothèque PySerial :

```
1 line = serial.readline().decode()
```

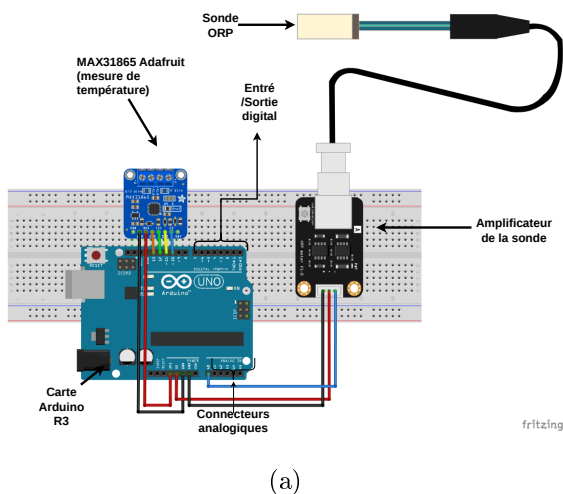


FIGURE 2 – (a) Schéma de montage des éléments de la sonde, la sonde de température PT100, non représentée, est reliée à la carte d'amplification MAX31865 par les borniers. (b) Photographie du boîtier monté : le capteur thermique étant à gauche et la sonde ORP en haut de la figure.

4.3 Connexion boîtier / Ordinateur

Le code contenu dans le dossier "SRC/Mesure.py" (voir A.1) contient une fonction permettant d'établir la connexion série avec l'Arduino. L'utilisateur peut choisir de fournir un port manuellement, sinon, grâce à une reconnaissance de nom de port, le programme va sélectionner automatiquement le nom relié à l'Arduino. Un filtre pour reconnaître les ports Linux ou Windows a été mis en place et présenté dans l'annexe A.3.

5 Calibration

5.1 L'équation d'oxydo-réduction :

Le boîtier Arduino renvoie un potentiel mesuré (V_m en mV) qui est relié par une relation affine au potentiel réel (E_r en mV) de la solution selon l'équation :

$$E_r = a \cdot V_m + C_0 \quad (3)$$

a : pente ; C_0 : ordonnée à l'origine ; V_m : Potentiel mesuré ; $E_r = E_h$: Potentiel réel correspondant au potentiel référencé à l'ESH (voir 3)

C_0 ² représente aussi l'*offset*, c'est-à-dire le coefficient correcteur qui va permettre de rectifier les erreurs de précision du boîtier et de la sonde ORP. La pente a représente le gain de la sonde, c'est le rapport entre ce qu'elle mesure et ce qui existe réellement.

5.2 Calibration d'usine et à 1 étalon

La calibration d'usine suppose que l'appareil est déjà calibré ($C_0 = 0$), c'est une valeur idéale qui n'est jamais réelle. Pour une calibration à un étalon, en reprenant l'équation (3) et en considérant les informations³ du fabricant des sondes.

$$C_0 = E_r - ((2 - i) \cdot 1000) = E_r - a \cdot V_m$$

Exemple : Si une sonde ORP est plongée dans une solution calibrée à 240 mV ($= E_r$) et que $V_m = 230$ mV alors $C_0 = 10$.

5.3 Calibration à deux étalons

Pour une calibration à deux étalons, l'utilisateur devra effectuer deux mesures de potentiel mesuré (V_m) dans deux solutions calibrées différentes pour calculer deux potentiels réels (E_{r1} et E_{r2}). Ensuite une fonction du code va tracer une courbe de tendance à l'aide de la pente calculée (a) en utilisant la fonction **np.poly1d**. Cette fonction nous renvoie donc la courbe de tendance avec la pente (a), l'offset (C_0) et R^2 ⁴ le coefficient de détermination. On calcule la pente (a) et l'offset (C_0) de la manière suivante :

$$a = \frac{E_{r2} - E_{r1}}{V_{m2} - V_{m1}} \quad (4)$$

La courbe de tendance étant une droite, il suffit de faire :

$$C_0 = E_{r1} - a \cdot V_{m1} = E_{r2} - a \cdot V_{m2} \quad (5)$$

Voici un exemple d'utilisation, réalisé à partir des deux solutions calibrées du laboratoire : 240 mV et 470 mV de Hanna Instrument.

Dans cet exemple, la pente ($a = 1.06$) signifie pour 1 mV réel, la sonde mesure 1.06 mV.

2. Ce coefficient correctif est obtenu en effectuant la moyenne sur un nombre de mesures choisi par l'utilisateur.

3. i est la valeur brute lue par l'ADC de l'Arduino (en volts, typiquement entre 0 et 5V) ; 2 est la tension de référence au milieu de la plage de la sonde (valeur par défaut du fabricant). On suppose ici que $a = 1$.

4. Avec seulement 2 points, R^2 vaut toujours 1 car la droite passe (forcément) exactement par ces 2 points

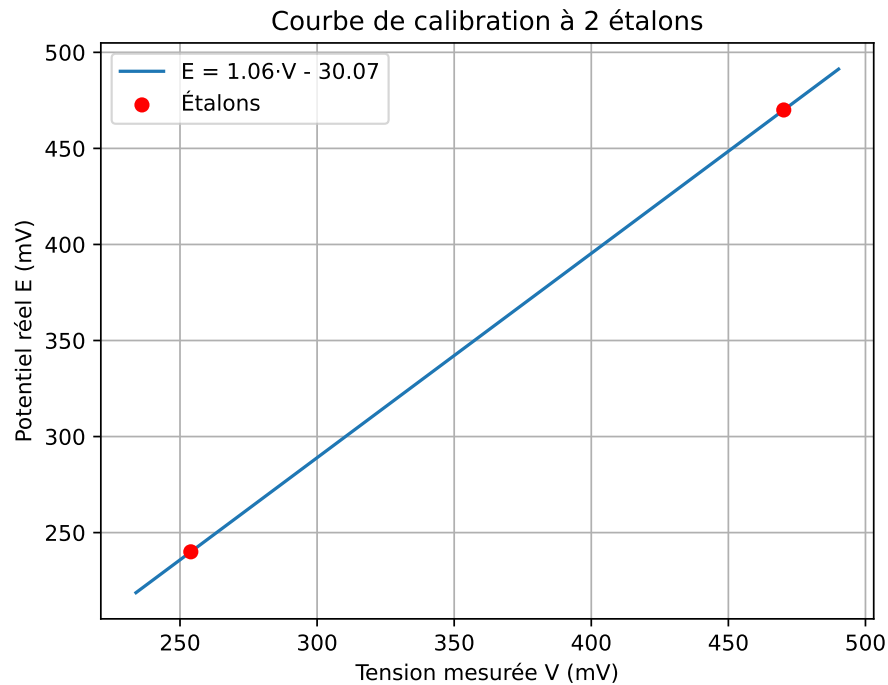


FIGURE 3 – Courbe de tendance d’une calibration à deux étalons réalisée à partir de deux solutions calibrées : une à 240 mV et l’autre à 470 mV.

Ainsi, au premier ordre on peut quand même utiliser la calibration à un étalon en considérant $a = 1$ mais celle à deux étalons permet de corriger le gain de la sonde en plus du coefficient correctif (*l’offset*). Par ailleurs, les résultats de cette calibration étaient :

- $V_{m1} = 470.200$ pour la solution étalon à 470 mV
- $V_{m2} = 253.939$ pour la solution étalon à 240 mV

Et en vérifiant :

Point 1 (étalon 240) : $1.0635 \times 253.94 - 30.07 \approx 240$ mV

Point 2 (étalon 470) : $1.0635 \times 470.20 - 30.07 \approx 470$ mV

6 Les mesures

6.1 Acquisition des Mesures

Une fois la sonde ORP plongée dans la solution souhaitée, l'utilisateur peut :

1. obtenir sous forme de graphe les mesures de températures et de potentiels en direct (la figure s'actualise en temps réel), avec la possibilité d'enregistrer les données à partir d'un temps d'acquisition choisi pour avoir des mesures stables.
2. choisir le nombre de mesures et obtenir un graphe des températures et des potentiels.

La première méthode est utile car la stabilisation des mesures peut être longue. Toutefois la deuxième, nécessitant moins de ressources à Python, est à privilégier à Madagascar.

6.2 Tests des sondes ORP

6.2.1 Méthode d'analyse

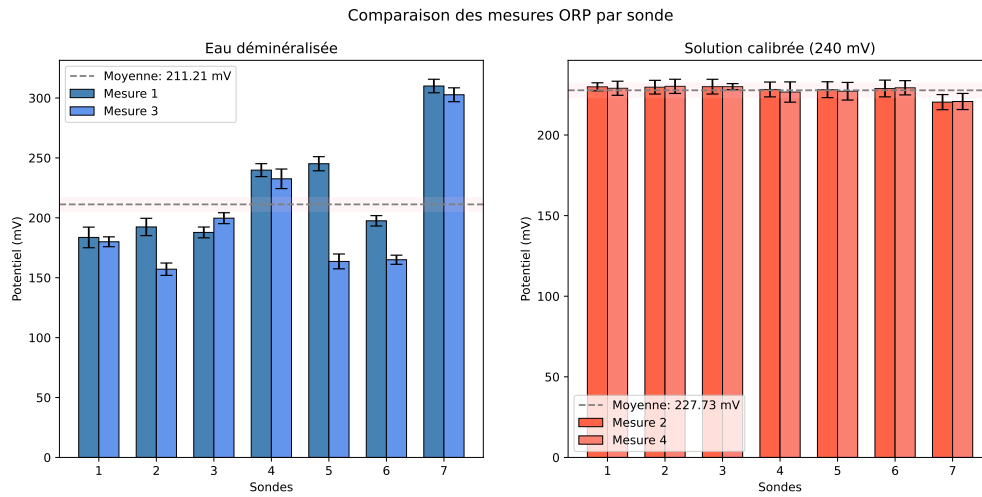
Pour tester la reproductibilité des mesures de nos sondes, nous avons effectué 4 mesures par sonde en faisant des allers-retours entre une solution calibrée et de l'eau déminéralisée. Ce test a été réalisé avec deux solutions calibrées avec une calibration d'usine (voir 5.2) : à 240 mV (**Test 1**) et à 470 mV (**Test 2**).

6.2.2 Discussion des résultats

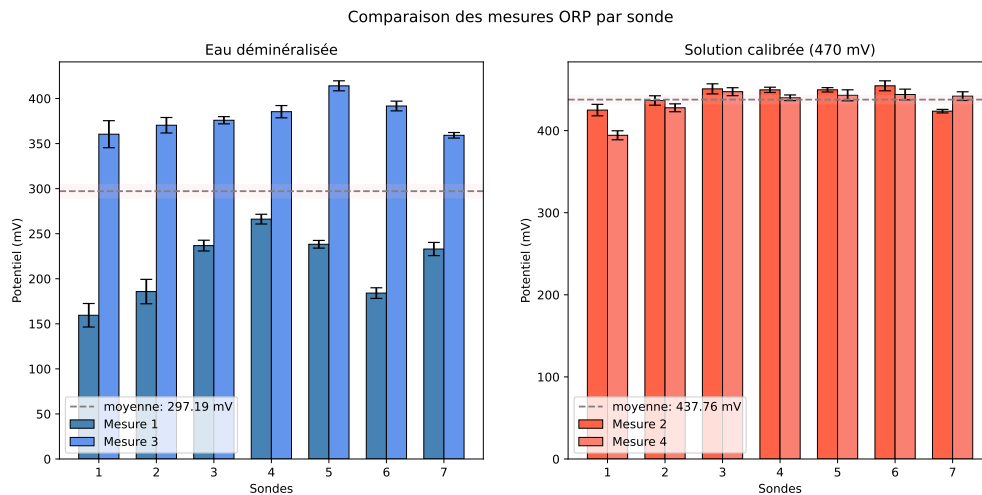
Reproductibilité dans la solution calibrée

À 240 mV (figure 4a) et à 470 mV (figure 4b) les sept sondes affichent des valeurs très groupées autour de la moyenne de 227,73 mV et 437,66 mV avec des écarts-types moyens faibles ($\sigma_{m,240} = 4.47$ et $\sigma_{m,470} = 5.01$). L'*offset* mesuré moyen du test 1 est de $-12,27$ mV⁵ tandis que l'*offset* moyen du test 2 est de $-32,34$ mV. En théorie, l'*offset* devrait être constant mais ces tests étant réalisés avec une calibration d'usine ($a = 1$), en reprenant l'exemple de calibration à deux étalons (5.3), la pente réelle implique que $C_0 = E_r - a \cdot V_m$ et si $a > 1$ alors plus V_m est grand plus le terme $a \cdot V_m$ s'écarte de E_r et C_0 devient plus négatif. La reproductibilité inter-sonde est donc très bonne dans des solutions tampons, explicable par leur potentiel bien défini permettant une stabilisation rapide de la sonde.

5. Cet *offset* est cohérent avec la précision indiquée par les fabricants des sondes qui est de ± 10 mV



(a) Test 1



(b) Test 2

FIGURE 4 – Comparaison des mesures ORP par sonde. Les mesures alternent entre eau déminéralisée (E) et solution calibrée (C) selon l'ordre : E(1) → C(2) → E(3) → C(4), où le chiffre entre parenthèses indique le numéro de mesure. (a) Test sur la solution calibrée à 240 mV. (b) Test sur la solution calibrée à 470 mV.

6.2.3 Reproductibilité dans l'eau déminéralisée

De manière générale, après stabilisation des sondes, l'écart-type intra-sonde moyen sur les 2 tests est faible ($\sigma_{intra,m} = 6,545$).

À 240 mV (figure 4a) : les mesures dans l'eau déminéralisée sont très dispersées entre les sondes (de ~ 160 mV à ~ 310 mV pour la sonde sept), avec une moyenne de 211,21 mV. Contrairement à la solution calibrée, l'écart-type moyen inter-sonde est très élevé ($\sigma_{inter240} = 45,51$), cependant, la différence de potentiel entre les mesures 1 et les mesures 3 reste assez similaire $\Delta_{1,3} = 16.3$ ⁶. La reproductibilité reste acceptable, à l'exception de la sonde 5.

À 470 mV (figure 4b) : la dispersion des valeurs est encore plus marquée. En effet la différence moyenne

6. Sonde 5 non considérée

entre 2 mesures dans l'eau déminéralisée est de $\Delta_{3-1} = 164.78$ mV. On remarque cependant que chaque mesure 3 des sept sondes convergent vers un même potentiel ($\sigma_{inter,mesure3} = 17,94$). Cette différence peut être expliquée par le passage dans une solution à 470 mV qui provoquerait une inertie très importante pour la sonde. En effet, l'eau étudiée étant peu tamponnée et avec une faible force ionique serait très sensible à des traces d'oxydants, contamination de la solution précédente⁷. ou encore nécessiterait une longue stabilisation⁸

7 Conclusion

Le stage a porté ses fruits et ses objectifs ont été atteints. Le code Python a été restructuré, l'appareil a été automatisé et testé. Pour réaliser des mesures précises, il est préférable de connaître la plage de potentiel du milieu étudié avant toute calibration. L'influence d'une solution de plus haut potentiel que le milieu sur les sondes suggère que la calibration à deux étalons est peu adaptée à ce type de mesure. Néanmoins, la précision de ces sondes est suffisante pour les travaux pratiques dans le milieu universitaire. Aujourd'hui, l'ensemble des connaissances et apports de cet appareil vont pouvoir être partagés avec des enseignants malgaches, ce qui est à la fois une grande opportunité et fierté.

8 Remerciements

En premier lieu, je tiens à remercier chaleureusement M. François MÉTIVIER, enseignant-chercheur à l'IPGP. En tant que tuteur de stage, il m'a partagé nombre de ses connaissances, notamment en programmation (Python) et en électronique (Arduino), mais il m'a aussi permis de découvrir l'approche scientifique lorsque l'on conçoit un appareil : comment s'assurer qu'il fonctionne correctement, les questions et tests que l'on peut utiliser pour en délimiter l'utilisation. De plus, je tiens à le remercier pour avoir proposé à trois de mes camarades de CPES et moi-même de participer au projet SAFE-M, qui se traduira par un voyage pédagogique à Madagascar en août 2026.

Par ailleurs, je remercie l'Institut de Physique du Globe de Paris pour m'avoir accueilli au sein de son bâtiment. C'est un endroit très agréable pour travailler et j'ai pu y rencontrer des chercheurs et des stagiaires (doctorants et étudiants en master) fort sympathiques, avec qui j'ai pu avoir des discussions intéressantes sur les conditions et les particularités du monde de la recherche.

Enfin, je remercie mes parents pour m'avoir apporté une lecture extérieure de mon rapport, ainsi qu'une correction des fautes d'orthographe et de syntaxe.

Références

- [1] MÉTIVIER François. *Mesures automatisées en physique et chimie de l'environnement*. Cours de Licence 3 CPES, Université Paris Cité / IPGP, 2025. Notes de cours.
- [2] RODIER Jean et LEGUBE Bernard, *L'analyse de l'eau, contrôle et interprétation*. 10^e édition. Lieu d'édition : DUNOD, 2026 (1^{re} édition en 1959), 1759 pages. **voir p. 121, § 5.4**
- [3] STRIGGOW Brian. *Field Measurement of Oxidation-Reduction Potential (ORP)*, OPERATING PROCEDURE, Janvier 2013. **voir p. 16, § 6.5.3.A**

7. Un test a montré que la sonde se calibre bien après un passage dans la solution à 470mV dans la solution à 240 mV.

8. Une sonde a été laissée pendant plusieurs heures, après un passage dans la solution à 470 mV, sans jamais redescendre jusqu'aux valeurs de la mesure 1.

- [4] LOUAULT Cyril, professeur de Janson de Sailly *Chapitre 3 : Étude thermodynamique des réactions d'oxydo-réduction, Documents d'accompagnement*. Cours de Chimie CPES 3, 2025-2026. Notes de cours.
- [5] WANG, Xiaodong *et al. Characterization of Oxidation-Reduction Potential Variations in Biological Wastewater Treatment Processes : A Study from Mechanism to Application*. Processes, 2022, vol. 10, n°12, article 2607. Disponible sur : <https://www.mdpi.com/2227-9717/10/12/2607>
- [6] YIN, XIN *et al. Identifying redox transition zones in the subsurface of a site with historical contamination. Science of The Total Environment*, 2021, vol. 762, p. 143105. Disponible sur : <https://www.sciencedirect.com/science/article/pii/S0048969720366353>

Autres références :

Outils Github

- [7] Hello World (Introduction à Github). Disponible sur : <https://docs.github.com/en/get-started/start-your-journey/hello-world>.
- [8] Git docs pour : Migrations : Importing a repository with GitHub Importer. Disponible sur <https://docs.github.com/en/migrations/importing-source-code/using-github-importer/importing-a-repository-with-github-importer>

Outils Git

- [9] Gittutorial, Reference, (17/12/2014 , 20/04/2026). Disponible sur <https://git-scm.com/docs/gittutorial>
- [10] Install, Windows (17/12/2014, 17/04/2026). Disponible sur <https://git-scm.com/install/windows>
- [11] Git docs pour :
 - *Git add*(17/12/2014,20/04/2026) disponible sur <https://git-scm.com/docs/git-add/fr>
 - *Git pull* (17/12/2014,20/04/2026) disponible sur <https://git-scm.com/docs/git-pull/fr>

Outils Overleaf

- [12] How to change paragraph spacing in L^AT_EX. Disponible sur https://fr.overleaf.com/learn/latex/Articles/How_to_change_paragraph_spacing_in_LaTeX
- [13] Headers and footers. Disponible sur https://www.overleaf.com/learn/latex/Headers_and_footers
- [14] *How to Write a Thesis in LaTeX pt 3 - Figures, Subfigures and Tables*. Disponible sur <https://www.youtube.com/watch?v=mYfSRmMduGo&t=24s>
- [15] Le rapport de Licence. Template écrit par M.Métivier.

A Annexes

A.1 Organisation du code

```
SAFE-M-ORP (1)
├── Data
│   ├── data_figures
│   ├── data_(T,V)
│   └── data_calibration
├── documentation
│   └── Documentation.md
├── SRC
│   ├── __pycache__
│   ├── arduino_redox
│   ├── calibration.py
│   ├── consigne.py
│   └── Mesure.py
├── .gitattributes
└── README.md
```

Le projet SAFE-M-ORP (1) est constitué en deux blocs principaux :

1. Le dossier Source (*SRC*) où l'on retrouvera :
 - *Mesure.py* qui contient toutes les fonctions pour permettre d'effectuer les mesures, les graphiques, les enregistrements (voir [A.3](#)) et la connexion au port Arduino.
 - *calibration.py* qui contient les fonctions permettant un étalonnage d'usine, à 1 et 2 étalons (voir [A.3](#)) ainsi que les fonctions pour enregistrer ces calibrations.
 - *consigne.py* qui contient le menu interactif avec l'utilisateur lui permettant de choisir ce qu'il veut faire.
 - *arduino_redox.py* qui contient le code C/C++ à téléverser sur les cartes Arduino Uno (cf [4.2](#)) pour l'acquisition des mesures.
 - *__pycache__* est un dossier généré automatiquement par Python qui contient les fichiers compilés (.pyc) des scripts.
2. Le dossier Data qui contient les données des mesures (et calibration) sous la forme fichier *.csv* ou des graphiques sous la forme *.png*. Les fichiers *.csv* sont principalement organisés comme ci-dessous :

```
1 # Donnees Temperature / Potentiel(calibré), pour V : écart type =5.74 et
   moyenne = 53.14
2 21.61,260.00
3 21.61,250.00
4 21.64,250.00
5 21.61,250.00
6 21.61,250.00
```

Ainsi que 3 autres fichiers indépendants :

- Le *README.md* qui présente les caractéristiques essentielles de ce projet ainsi que les indications pour démarrer le programme.
- *Documentation.md* qui est un manuel d'utilisation pour l'utilisateur. Il explique comment est organisé le code ainsi que les remarques ou les conseils sur l'utilisation de l'appareil.
- *.gitattributes* normalise les fins de ligne du dépôt : il force les fichiers Python, Markdown et CSV à utiliser le format Unix (LF) permettant l'utilisation du code sur Windows ou Linux.

A.2 Utilisation de l'IA

Une grande partie de ce stage était de la programmation et soucieux d'apprendre de nouvelles compétences en Python et de manière générale dans l'environnement numérique (Github, Git...); l'IA a été sollicitée dans une approche pédagogique. L'usage de l'IA (principalement Claude) a consisté à demander des informations et des méthodes pour réaliser les actions voulues. Principale utilisation avec exemple :

- **Demander des explications après avoir lu de la documentation.**

Exemple : après avoir lu la documentation [9]

PROMPT : explique moi git diff – cached j'ai pas trop compris, voici ma doc

Modify some files, then add their updated contents to the index :

```
git add file1 file2 file3
```

You are now ready to commit. You can see what is about to be committed using git diff with the –cached option :

```
git diff –cached
```

(Without –cached, git diff will show you any changes that you've made but not yet added to the index.) You can also get a brief summary of the situation with git status :

```
3 juin
```

```
git diff –cached
```

Réponse Claude : Git a 3 zones qu'il faut visualiser :

Tes fichiers / L'index (staging) / Le dépôt Git

(disque) ==> (zone tampon) ==> (historique)

.....git add.....git commit

git diff (sans option) → compare disque vs index Ce que tu as modifié mais pas encore fait git add

git diff –cached → compare index vs dernier commit Ce qui est prêt à être commité (ce que tu as déjà fait git add)

- **Demander des outils python pour réaliser les fonctions/ codes désirés**

Exemple : On cherchait à corriger le code du graphique en live ;

PROMPT : j'ai essayé sur ma fonction graphe live de faire un filtre pour éviter que les erreurs de mesures (pic à 1300, –4000...) rende illisible le graph (avec V_{err}) mais depuis que j'ai écrit ça, le graphe est très peu fluide et ne marche pas bien :

```
1 def graphe_live($C0, s$):
2     (...)
3     return T, V ,fig
```

Réponse Claude : Le problème vient de ta boucle de filtrage : à chaque itération de la boucle principale, tu recopies tous les éléments valides de V_{err} dans V , ce qui fait que V grossit de façon incorrecte (doublons, triple counts...) et que la boucle for devient de plus en plus longue au fil du temps. (...) correction de la fonction

- **Demander de faire des recherches pour avoir des sources rapidement afin de gagner du temps** : Exemple : On cherchait des informations sur la composition de solution étalon suite à des problèmes avec les nôtres. **PROMPT** : trouve moi sur internet des solutions test de potentiel pour des ORP pour gold et platinum electrode, il me faut des informations précises sur ces solution (pas comme celles de HANNA INSTRUMENTS) **Réponse Claude** :Voilà une liste concrète de fournisseurs sérieux (hors Hanna) avec les produits disponibles à l'achat :

Solutions ORP à acheter — fournisseurs sérieux
 Reagecon — le plus complet
 Fabricant irlandais spécialisé, gamme ORP très large avec certificats d'analyse (COA) et fiches de sécurité (SDS). Précision ± 5 mV à 25 °C vs électrode Ag/AgCl Pt. Disponibles en 250 mL ou 500 mL. CymitQuimica
 RéférenceValeur à 25 °CURL directeRS200200 mVreagecon.com/en-gb/reagecon-200-mv-redox (...)
 Laboratoires Humeau — fournisseur français
 URL : humeau.com → Solutions étalon → Solutions étalon Redox
 Fisher Scientific France — revendeur multi-marques

A.3 Quelques exemples de fonction du projet SAFE-M-ORP

```

1 \textbf{Fonction permettant la connexion automatique au boîtier.}\\
2 Une partie de la fonction a été tronquée (...) pour mettre en avant le filtre
  Linux et Windows
3 def connexion_port(br=115200, portIN=''):
4     #(...)
5     # Détection automatique
6     ports = list(serial.tools.list_ports.comports()) #liste tous les ports sé
      rie disponibles sur l'ordinateur.
7
8     for port in ports:
9         p = str(port)
10        # Windows
11        if 'Arduino' in p or 'Périphérique_série' in p or 'série_USB' in p :
12            portIN = port.device
13            break
14        # Linux
15        if 'ttyACM' in port.device or 'ttyUSB' in port.device:
16            portIN = port.device
17            break
18    #(...)
19    try: #tentative de connexion au port détecté
20        s = serial.Serial(port=portIN, baudrate=br, timeout=5)
21        print(f'Connexion_réussie_sur_{portIN}')
22        return portIN, s
23    except serial.SerialException as e:
24        print(f"Erreur:_impossible_d'ouvrir_{portIN}:_{e}")
25    return '', 'error'

```

Exemple d'enregistrement des données sous format csv :

```

1 def enregistrement_csv (T,V_real,moy = None,sigma=None) :
2     """_summary_
3     Enregistre_les_données_de_température_et_de_potentiel_calibré_dans_un_fichier

```

```

4 CSV_horodaté, accompagnées de la moyenne et de l'écart-type des tensions.
5 Le fichier est sauvegardé dans le dossier Data/data_(T,V) /.
6
7 Args:
8     T(list[float]): Liste des températures mesurées (en °C).
9     V_real(list[float]): Liste des potentiels calibrés (en mV).
10    moy(float, optional): Moyenne des tensions calibrées. Calculée
        automatiquement
11    si non fournie.
12    sigma(float, optional): Écart-type des tensions calibrées. Calculé
13    automatiquement si non fourni.
14
15 Returns:
16    str: Message de confirmation d'enregistrement.
17    """
18    BASE = Path(__file__).parent.parent
19    now = datetime.datetime.now()
20    nom_fichier = now.strftime("Mesure_Température_et_Voltage_%d_%B,_%Hh%M. csv"
        )
21    chemin = BASE/"Data"/'data_(T,V)'/nom_fichier
22    n = min(len(T), len(V_real))
23    if moy is None:
24        moy = np.mean(V_real[:n])
25    if sigma is None:
26        sigma = np.std(V_real[:n])
27    resultat = np.column_stack((T[:n], V_real[:n]))
28    with open(chemin, 'w') as f:
29        np.savetxt(f, resultat, delimiter=',', fmt='%.2f', header=f'Donnees_
        Temperature_/Potentiel(calibré), pour V: écart_type={sigma} et
        moyenne={moy}')
30    return 'Le fichier csv a bien été enregistré.'

```

Fonction permettant la calibration à deux étalons (la fonction représentant le graphique n'est pas présentée ici.)

```

1
2 def calibration_2_etalons(s, E1=None, E2=None, V1=None, V2=None) :
3     """_summary_
4     Effectue une calibration à deux étalons par régression linéaire.
5     Demande à l'utilisateur les potentiels des deux solutions étalons, acquiert
6     les mesures sur chacune d'elles, puis calcule la droite de tendance par
7     régression linéaire (scipy.stats.linregress) entre les tensions moyennes
8     mesurées et les potentiels théoriques.
9     Args:
10    s(serial.Serial): Objet de connexion série avec l'Arduino.
11    E1(float, optional): Potentiel théorique de la solution étalon 1 (en mV).
12    E2(float, optional): Potentiel théorique de la solution étalon 2 (en mV).
13    V1(list[float], optional): Tensions brutes mesurées sur l'étalon 1.
14    V2(list[float], optional): Tensions brutes mesurées sur l'étalon 2.
15    V_real1: tension convertie en mV
16    V_real2: tension convertie en mV
17    E1 et E2 sont demandés à l'utilisateur si non fournis, V1 et V2 sont acquises
        via mes.data() si non fournies
18    V_real1 et V_real2 sont acquises via V1 et V2
19
20 Returns:
21    tuple: (C0, V1_moy, V2_moy, tendance, r_squared, E1, E2)

```

```

22  _____C0_(float):_Intercept_de_la_droite,_utilisé_comme_offset_de_
    calibration_(en_mV).
23  _____V1_moy_(float):_Tension_moyenne_mesurée_sur_l'étalon_1_(en_mV).
24  _____V2_moy_(float):_Tension_moyenne_mesurée_sur_l'étalon_2_(en_mV).
25  _____tendance_(np.poly1d):_Droite_de_calibration_sous_forme_de_polynôme.
26  _____r_squared_(float):_Coefficient_de_détermination_R^2_de_la_régression.
27  _____E1_(float):_Potentiel_théorique_de_l'étalon_1_(en_mV).
28  _____E2_(float):_Potentiel_théorique_de_l'étalon_2_(en_mV).
29  """
30      if E1 is None and V1 is None:
31          E1 = float(input('Potentiel_de_la_solution_étalon_1(mV):_'))
32          input('Placer_votre_sonde_dans_la_solution_étalon_1,_quand_vous_êtes_prêt,écrivez_OK==>')
33          print('Début_des_mesures...')
34          T, V1 = mes.data(s, N=100)
35          print('Fin_des_mesures')
36      if E2 is None and V2 is None:
37          E2 = float(input('Potentiel_de_la_solution_étalon_2(mV):_'))
38          input('Nettoyez_et_séchez_votre_sonde,_puis_placez_la_dans_la_solution_étalon_2,_quand_vous_êtes_prêt,écrivez_OK==>')
39          print('Début_des_mesures...')
40          T, V2 = mes.data(s, N=100)
41          print('Fin_des_mesures')
42
43      V1_moy = np.mean(V1)
44      V2_moy = np.mean(V2)
45
46      # Conversion en mV
47      V1_real = (2 - V1_moy) * 1000
48      V2_real = (2 - V2_moy) * 1000
49      a = (E2 - E1) / (V2_real - V1_real)
50      C0 = E1 - a * V1_real
51      tendance = np.poly1d([a, C0])
52      return a, C0, E1, E2, V1_real, V2_real, tendance

```